

Comparing Graph-Based and Traditional Machine Learning for Malware Detection

Alex Klee
Virginia Tech
alexk11@vt.edu

1 Academic Integrity Pledge

I wrote this report myself, not using generative AI technologies.

2 Abstract

This paper compares traditional machine learning using feature extraction with graph neural network representation learning for the task malware classification using function call graph data. Two graph-based models and three classical models are trained on the MalNet-Tiny dataset, Random Forest, SVM, and Gradient Boosting using extracted features, and GCN and GraphSAGE trained on the raw graph structure. GraphSAGE achieves the best overall performance at 93.1% accuracy, while the classical models remain competitive with scores above 85% trained on only 5000 examples. These results suggest that the learned representations provide meaningful gains over traditional machine learning when malware is represented as graphs, while the models using handcrafted features are more convenient and provide competitive results.

3 Problem Statement and Goal

Malware detection has been a difficult problem in cybersecurity for a long time, and for a while it has been reliant on machine learning approaches operating on static features extracted from executable files. More recently, graph-based learning has been used for malware classification, since malware executable files have proved to have a useful structure when represented as function call graphs. This format captures the structural representation of the program's flow, and allows the models to capture meaningful patterns that feature vectors may not be able to fully represent. When only graph representations are available of potential malware files, it becomes a question of how to best discover the intention of the file, benign or malicious, and the construction of a classifier becomes important.

This work evaluates whether graph neural networks that learn representations directly from graph topology outperform classical machine learning models trained on handcrafted graph statistics, under a controlled experiment on the MalNet-Tiny dataset.

4 Related Work

Malware detection has traditionally used feature extraction from executable files to make classifications, and there are many examples of that being successful in the literature. One example of this is this is the EMBER dataset, which created a benchmark for this approach by providing pre-extracted features from over one million Windows PE files and showing that machine learning techniques including gradient boosting can achieve strong performance on binary malware classification. ([Anderson and Roth, 2018](#)).

Graph representations have also been used in the literature for malware detection, with function call graphs being introduced as capturing structural patterns and internal relationships that are more resistant to some obfuscation techniques than raw byte features ([Hu et al., 2009](#)). Since then, graph neural networks have become a strong option and a natural fit for learning from this kind of structured data. Hamilton et al. extended that idea by building GraphSAGE, a graph learning framework that aggregates neighbor features and has strong performing generalizing to unseen graphs ([Hamilton et al., 2017](#)). The MalNet dataset was introduced specifically for research on graph representation learning for malware detection, and it provides over 1.2 million function call graphs across 47 different types of malware and 696 families ([Freitas et al., 2021](#)). The paper that introduced MalNet notes that baseline methods can be effective at scale, which motivates the comparison done in this work. Bilot et al. provide a survey of different approaches for graph representation learning for malware detec-

tion, as well as different how CFGs (control flow graphs) vs. FCGs (function call graphs) perform. This research reinforces the sentiment that GNNs consistently have competitive results for malware detection. (Bilot et al., 2023).

5 Approach

This work evaluates two separate classification pipelines on the same graph dataset to compare the performance of the graph representation learning methods vs. the traditional classifiers. Both training pipelines use the same train, validation, and test splits of the MalNet-Tiny dataset, which helps keep the experiment consistent. Five different strategies are evaluated, with three classical models: Gradient Boosting, Support Vector Machine, and Random Forest, and two graph models, a Graph Convolutional Network and a GNN using GraphSAGE. To train the standard classifiers, features are extracted from the graph data containing various information about graph statistics. For the GNNs, the graphs are fed directly, with node-level features added to help the models learn richer representations. After training, all models will be independently evaluated on the test set, using accuracy and macro F1, where the macro F1 is weighted equally across all classes. The per-family performance will also be captured, to see if the different models may perform better on different types of malware.

The core contribution of this work is the direct comparison between the two different representation strategies on the data. This setup addresses the question of how much that learned representation adds over handcrafted graph statistics when only graph data is available. A limitation of this comparison is that it only the graph data was available, and a stronger comparison would evaluate the classical ML using features directly from the executable files.

6 Threat Model

The threat model assumes an attacker distributing some types of malware, to match this data the attack could look like malicious Android APKs through app stores or third-party distribution channels. The defender runs a classifier that extracts function call graphs from APKs and classifies them as benign or as malware, in one of 4 malware families. Raw binary files are not assumed to be available, only the extracted graph representations. The primary

security concern for a system like this is false negatives, which would be malware samples that are classified as benign, meaning the dangerous file evades the detection system. Adversarial attacks that modify the graph structure itself are a newer attack vector, with examples being function injection or call graph obfuscation. These are considered out of scope for this work but represent an important future direction.

7 Experimental Setup and Work Performed

7.1 Dataset

This experiment uses the MalNet-Tiny dataset, which is small subset of the full MalNet database containing 5,000 Android APK function call graphs across five classes. The tiny version of MalNet has five different malware families: AdDisplay, Adware, Benign, Downloader, and Trojan (Freitas et al., 2021). The dataset provides predefined train, validation, and test splits, which are used in training for all of the models. The graphs are generated from the function call structure of an Android application, meaning the nodes correspond to functions and edges represent calls between them. The MalNet dataset has no default node features,

7.2 Classical ML

To keep the experiment consistent, all three classical models are trained the same way. Before training, a feature vector the graph statistics is extracted from each graph using NetworkX. 21 features are captured, ranging from different structural properties like node and edge count, degree properties, skewness, connection statistics. These features were chosen with the goal of capturing both local and global properties without overloading the classifiers with features. Feature extraction is performed on the undirected version of each graph, and then all features are normalized before training. Three classical models are trained: Random Forest, Support Vector Machine, and Gradient Boosting. The Random Forest setup uses class balanced weights and searches over number of estimators, maximum depth, minimum samples per leaf, and maximum features. The SVM searches over both RBF and polynomial kernels, and Gradient Boosting searches over number of estimators, learning rate, maximum depth, subsample ratio, and minimum samples per leaf.

7.3 GNNs

To train the GNNs, the graphs are processed in their directed form with four features computed for each node, in-degree, out-degree, total degree, and the ratio of the first two. These features serve as the input to the GNN layers, which learn the graph representations through message passing. These features are different from the ones used in classical machine learning, since instead of summarizing the entire graph into a fixed feature vector they provide local context at each node used to help the GNN learn richer representations. In this work, two GNN architectures are evaluated. First, a Graph Convolutional Network is trained using three convolutional layers with improved self-loops. Residual connections are added to the GCN between layers along with normalization to stabilize training, since the malware graph data has the tendency to have skewed degree distributions. The GraphSAGE model is trained with three SAGEConv layers, each followed by a batch normalization. Both models use a combination of mean and max global pooling to produce the graph embeddings, and both GNNs are trained with the AdamW optimizer, a learning rate schedule, cross entropy loss, and early stopping based on validation F1 scores with a patience of 20 epochs.

7.4 Evaluation

All five models are evaluated on the same predefined test split from MalNet-Tiny using accuracy and macro F1 as the main metrics. Macro F1 is used here because it weights each class equally regardless of the count, which is important for evaluating performance across different families that may differ in size. Figure 1 presents the test accuracy and macro F1 for all five models.

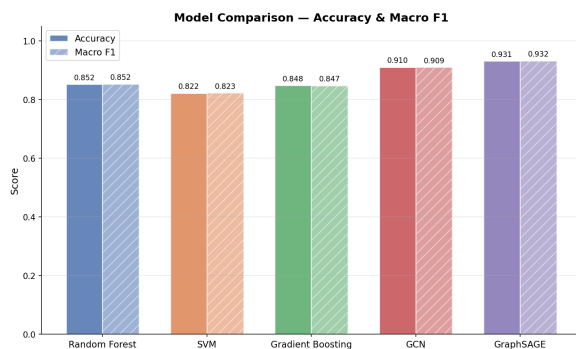


Figure 1: Test accuracy and macro F1

GraphSAGE achieves the best performance at 93.1% accuracy and 93.2% macro F1, followed

by GCN at 91.0% and 90.9%. Out of the classical, models Random Forest achieves the best score with 85.2%, followed by Gradient Boosting at 84.8% and SVM at 82.2%.

The heatmap in figure 2 presents the per-family F1 breakdown: Adware is the most difficult class

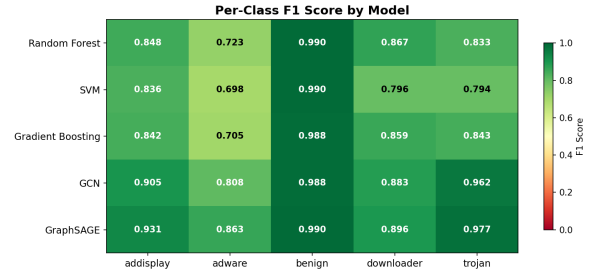


Figure 2: Per-Family Breakdown

for every model, with the best model achieving only 86.3% F1, which makes sense because out of those 4 malware families, adware is the has the highest chance to resemble a benign program at the structural level. Benign is the easiest class across all models, with all five achieving at least 98.8% F1, a very low false negative rate, which is important from a security perspective. The trojan family has the biggest gap between the graph based and classical models, which suggests that trojans have distinguishable patterns in their function call graphs that are better captured through learning that representation directly. Gradient Boosting performs slightly worse than Random Forest despite typically being a stronger method, which suggests that the handcrafted feature vectors might not be well suited for a boosting algorithm due to something like the lack of structure or potential noise.

8 Findings and Implications

The most important finding is the GraphSAGE’s strong performance on the small dataset, demonstrating the effectiveness of graph representation learning for malware detection. While basic GCNs aggregate information from all neighbors equally during training, GraphSAGE actually learns a separate aggregation function, so it becomes less sensitive to nodes with very high degree (Hamilton et al., 2017). This is likely a big reason that GraphSAGE performs strongly on malware function call graphs, which tend to have highly skewed degree distributions due to the fact that a small number of functions are called by a large proportion of the graph, which are generally called ‘hub functions’ in reverse engineering. The aggregation step for

normal GCNs produces diluted representations for those hub nodes, but the GraphSAGE framework’s approach ends up limiting the influence of any single neighbor. Another important result is that the Random Forest model achieves 85.2% accuracy, only 8 percentage points behind GraphSAGE. This suggests that a substantial portion of the information relevant to malware classification is expressible through simple graph statistics. This tradeoff is important since Random Forest is much quicker both to train and evaluate and its much more interpretable with the feature statistics, while GNNs are known to struggle with interpretability. The per-family analysis backs up the idea that GNNs provide the most value on more distinctive structural families like trojan, where the call graph topology likely has some important patterns that are captured. The consistently high F1 for the benign files across all models is also an encouraging security finding, this means all five approaches had low false positive rates for malware, which is always nice from a security standpoint.

9 Conclusions

This work presents a controlled comparison of feature engineering and graph neural network representation learning for the task malware family classification on function call graphs. GNNs outperform classical ML by around 8 percentage points, with the best model GraphSAGE achieving 93.1% accuracy compared to 85.2% for the best classical model. This demonstrates that the graph neural networks learn more meaningful patterns of the program, but that margin is relatively small, so the classical ML techniques remain competitive options, especially when interpretability or computational efficiency are a priority. The main limitation of this comparison is that only the graph data is available, and a more complete comparison would be able to evaluate classical ML trained on PE files against graph based GNNs on the same raw samples, which would require access to binary files. Future work should aim to create that data in order to make this comparison, and other directions could include improving the GNN explainability with tools like GNNExplainer to help best understand the advantages of GNNs.

References

Hyrum S. Anderson and Phil Roth. 2018. [Ember: An open dataset for training static pe malware machine](#)

[learning models](#). *arXiv preprint arXiv:1804.04637*.

Tristan Bilot, Nour El Madhoun, Khaldoun Al Agha, and Anis Zouaoui. 2023. [A survey on malware detection with graph representation learning](#). *arXiv preprint arXiv:2303.16004*.

Scott Freitas, Yuxiao Dong, Joshua Neil, and Duen Horng Chau. 2021. [A large-scale database for graph representation learning](#). In *NeurIPS Datasets and Benchmarks Track*.

William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. [Inductive representation learning on large graphs](#). In *Advances in Neural Information Processing Systems (NeurIPS)*.

Xin Hu, Tzi-cker Chiueh, and Kang G. Shin. 2009. Large-scale malware indexing using function-call graphs. In *Proceedings of the 16th ACM Conference on Computer and Communications Security*, pages 611–620. ACM.